

Autonomyx Optimize — Production-Ready App Scaffold

Recommended Stack

- **Framework:** Next.js App Router
 - **Language:** TypeScript
 - **UI:** React + Tailwind + shadcn/ui
 - **Charts:** Recharts
 - **State:** React Query + local UI state
 - **Auth:** Clerk / Auth.js / enterprise SSO abstraction
 - **Forms:** React Hook Form + Zod
 - **Tables:** TanStack Table
 - **Mock API:** typed in-app repository layer first, swappable with real backend
 - **Permissions:** centralized RBAC hook + route guard helpers
-

Folder Structure

```
src/  
  app/  
    (auth)/  
      login/  
        page.tsx  
    (dashboard)/  
      layout.tsx  
      page.tsx  
      recommendations/  
        page.tsx  
        [id]/  
          page.tsx  
      approvals/  
        page.tsx  
        [id]/  
          page.tsx  
      executions/  
        page.tsx  
        [id]/  
          page.tsx  
      renewals/  
        page.tsx  
      open-alternatives/
```

Control Tower

```
    page.tsx
  policy-builder/
    page.tsx
  roles/
    page.tsx
  settings/
    page.tsx
  layout.tsx
  globals.css

components/
  app-shell/
    sidebar.tsx
    topbar.tsx
    tenant-switcher.tsx
    role-preset-switcher.tsx
  cards/
    metric-card.tsx
    savings-impact-card.tsx
    info-card.tsx
  charts/
    spend-trend-chart.tsx
    savings-category-chart.tsx
  recommendations/
    recommendation-table.tsx
    recommendation-list.tsx
    recommendation-detail-header.tsx
    recommendation-financial-panel.tsx
  approvals/
    approval-list.tsx
    approval-decision-panel.tsx
  executions/
    execution-job-list.tsx
    execution-plan-card.tsx
  renewals/
    renewals-table.tsx
  settings/
    integration-status-card.tsx
    security-posture-card.tsx
  shared/
    page-header.tsx
    risk-badge.tsx
    status-badge.tsx
    empty-state.tsx
    audit-log-table.tsx

features/
  control-tower/
```

```
    queries.ts
    selectors.ts
    types.ts
recommendations/
  queries.ts
  mutations.ts
  types.ts
approvals/
  queries.ts
  mutations.ts
  types.ts
executions/
  queries.ts
  mutations.ts
  types.ts
renewals/
  queries.ts
  types.ts
policy-builder/
  queries.ts
  mutations.ts
  types.ts
settings/
  queries.ts
  mutations.ts
  types.ts

lib/
  auth/
    current-user.ts
    guards.ts
    permissions.ts
    session.ts
  api/
    client.ts
    errors.ts
    response.ts
  mock/
    db.ts
    seed.ts
    repositories/
      recommendations-repo.ts
      approvals-repo.ts
      executions-repo.ts
      renewals-repo.ts
      settings-repo.ts
  rbac/
    roles.ts
```

```
    policies.ts
    use-permission.ts
    use-role-preset.ts
  utils/
    currency.ts
    dates.ts
    cn.ts

  hooks/
    use-tenant.ts
    use-page-filters.ts
    use-notifications.ts

  types/
    api.ts
    auth.ts
    common.ts
    domain.ts

  config/
    navigation.ts
    tenants.ts
    role-presets.ts
```

Core App Contracts

Domain Types

```
export type RiskLevel = "low" | "medium" | "high";
export type RecommendationStatus =
  | "drafted"
  | "awaiting_approval"
  | "in_review"
  | "approved"
  | "rejected"
  | "executed"
  | "rolled_back";

export interface Recommendation {
  id: string;
  title: string;
  category: "saas" | "cloud" | "llm" | "vendor" | "open_source_replacement";
  vendor?: string;
  team?: string;
```

```

    risk: RiskLevel;
    confidence: "low" | "medium" | "high";
    status: RecommendationStatus;
    projectedSavingsAnnual: number;
    implementationCost: number;
    paybackDays: number;
    issue: string;
    evidence: string[];
    skepticNotes: string[];
    rollbackPlan: string[];
    approvals: string[];
    impactArea: string;
  }

  export interface ApprovalItem {
    id: string;
    recommendationId: string;
    role: string;
    dueAtLabel: string;
    risk: RiskLevel;
    projectedSavingsAnnual: number;
    status: "pending" | "approved" | "rejected" | "escalated";
  }

  export interface ExecutionJob {
    id: string;
    recommendationTitle: string;
    mode: "controlled_live_run" | "manual_after_approval" | "rollback";
    status: "queued" | "running" | "completed" | "failed" | "rolled_back";
    owner: string;
    startedAtLabel: string;
    completedAtLabel?: string;
    steps: { label: string; done: boolean }[];
  }

```

Typed Mock API Layer

src/lib/api/response.ts

```

export interface ApiSuccess<T> {
  ok: true;
  data: T;
}

```

```
export interface ApiFailure {
  ok: false;
  error: string;
}

export type ApiResult<T> = ApiSuccess<T> | ApiFailure;
```

src/lib/mock/db.ts

```
import type { Recommendation, ApprovalItem, ExecutionJob } from "@types/domain";

export interface MockDb {
  recommendations: Recommendation[];
  approvals: ApprovalItem[];
  executions: ExecutionJob[];
}

export const mockDb: MockDb = {
  recommendations: [],
  approvals: [],
  executions: [],
};
```

src/lib/mock/repositories/recommendations-repo.ts

```
import { mockDb } from "@lib/mock/db";
import type { Recommendation } from "@types/domain";
import type { ApiResult } from "@lib/api/response";

export async function listRecommendations():
  Promise<ApiResult<Recommendation[]>> {
  return { ok: true, data: mockDb.recommendations };
}

export async function getRecommendationById(id: string):
  Promise<ApiResult<Recommendation>> {
  const found = mockDb.recommendations.find((item) => item.id === id);
  if (!found) return { ok: false, error: "Recommendation not found" };
  return { ok: true, data: found };
}
```

src/features/recommendations/queries.ts

```
import { useQuery } from "@tanstack/react-query";
import { getRecommendationById, listRecommendations } from "@lib/mock/
repositories/recommendations-repo";

export function useRecommendationsQuery() {
  return useQuery({
    queryKey: ["recommendations"],
    queryFn: listRecommendations,
  });
}

export function useRecommendationQuery(id: string) {
  return useQuery({
    queryKey: ["recommendation", id],
    queryFn: () => getRecommendationById(id),
    enabled: Boolean(id),
  });
}
```

Auth + RBAC Hooks

src/lib/rbac/roles.ts

```
export type AppRole =
  | "admin"
  | "cfo"
  | "procurement"
  | "engineering"
  | "it_admin"
  | "viewer";

export interface AppUser {
  id: string;
  name: string;
  email: string;
  roles: AppRole[];
  tenantId: string;
}
```

src/lib/auth/current-user.ts

```
import type { AppUser } from "@lib/rbac/roles";

export async function getCurrentUser(): Promise<AppUser> {
  return {
    id: "user_1",
    name: "Chinmay Panda",
    email: "user@example.com",
    roles: ["admin", "cfo"],
    tenantId: "t-1",
  };
}
```

src/lib/rbac/policies.ts

```
import type { AppRole } from "@lib/rbac/roles";

export const permissions = {
  viewControlTower: ["admin", "cfo", "procurement", "engineering", "it_admin", "viewer"],
  viewRecommendations: ["admin", "cfo", "procurement", "engineering", "it_admin"],
  approveActions: ["admin", "cfo", "procurement", "engineering", "it_admin"],
  executeActions: ["admin", "engineering", "it_admin"],
  editPolicies: ["admin"],
  manageSettings: ["admin"],
} satisfies Record<string, AppRole[]>;
```

src/lib/rbac/use-permission.ts

```
"use client";

import { permissions } from "@lib/rbac/policies";
import type { AppRole } from "@lib/rbac/roles";

export function hasPermission(userRoles: AppRole[], permission: keyof typeof permissions) {
  return userRoles.some((role) => permissions[permission].includes(role));
}
```


src/lib/auth/guards.ts

```
import { redirect } from "next/navigation";
import { getCurrentUser } from "@/lib/auth/current-user";
import { hasPermission } from "@/lib/rbac/use-permission";

export async function requirePermission(permission: Parameters<typeof
hasPermission>[1]) {
  const user = await getCurrentUser();
  if (!hasPermission(user.roles, permission)) {
    redirect("/");
  }
  return user;
}
```

Route-Based Pages

Dashboard Layout

src/app/(dashboard)/layout.tsx

```
import type { ReactNode } from "react";
import { Sidebar } from "@/components/app-shell/sidebar";
import { Topbar } from "@/components/app-shell/topbar";

export default function DashboardLayout({ children }: { children: ReactNode }) {
  return (
    <div className="min-h-screen bg-slate-50 text-slate-900">
      <div className="grid min-h-screen grid-cols-1 lg:grid-cols-[280px_1fr]">
        <Sidebar />
        <div>
          <Topbar />
          <main>{children}</main>
        </div>
      </div>
    </div>
  );
}
```

Control Tower Page

src/app/(dashboard)/page.tsx

```
import { requirePermission } from "@lib/auth/guards";
import { PageHeader } from "@components/shared/page-header";
import { MetricCard } from "@components/cards/metric-card";

export default async function ControlTowerPage() {
  await requirePermission("viewControlTower");

  return (
    <div className="space-y-6 p-6">
      <PageHeader
        title="Control Tower"
        description="Unified cost visibility, governed savings, and active risk
overview."
      />
      <div className="grid gap-4 md:grid-cols-2 xl:grid-cols-3 2xl:grid-cols-6">
        <MetricCard title="Total Spend" value="$4.8M" subtitle="-3.2% vs last
quarter" />
        <MetricCard title="Identified Savings" value="$682K"
subtitle="42 active recommendations" />
      </div>
    </div>
  );
}
```

Recommendation Detail Route

src/app/(dashboard)/recommendations/[id]/page.tsx

```
import { notFound } from "next/navigation";
import { requirePermission } from "@lib/auth/guards";
import { getRecommendationById } from "@lib/mock/repositories/recommendations-
repo";

export default async function RecommendationDetailPage({
  params,
}: {
  params: Promise<{ id: string }>;
}) {
  await requirePermission("viewRecommendations");
  const { id } = await params;
```

```

const result = await getRecommendationById(id);

if (!result.ok) notFound();

return <div className="p-6">{result.data.title}</div>;
}

```

Reusable UI Components

src/components/shared/page-header.tsx

```

interface PageHeaderProps {
  title: string;
  description?: string;
  actions?: React.ReactNode;
}

export function PageHeader({ title, description, actions }: PageHeaderProps) {
  return (
    <div className="flex flex-col gap-4 lg:flex-row lg:items-center lg:justify-between">
      <div>
        <h1 className="text-2xl font-semibold tracking-tight">{title}</h1>
        {description ? <p className="text-sm text-slate-500">{description}</p> : null}
      </div>
      {actions ? <div>{actions}</div> : null}
    </div>
  );
}

```

src/components/cards/metric-card.tsx

```

import { Card, CardContent } from "@components/ui/card";

interface MetricCardProps {
  title: string;
  value: string;
  subtitle?: string;
}

export function MetricCard({ title, value, subtitle }: MetricCardProps) {

```

```

return (
  <Card className="rounded-3xl border-0 shadow-sm">
    <CardContent className="p-5">
      <div className="text-sm text-slate-500">{title}</div>
      <div className="mt-2 text-2xl font-semibold tracking-tight">{value}</div>
    </CardContent>
  </Card>
);
}

```

Navigation Config

src/config/navigation.ts

```

import {
  LayoutDashboard,
  Lightbulb,
  CheckSquare,
  PlayCircle,
  FileText,
  RefreshCw,
  SlidersHorizontal,
  Users,
  Settings,
} from "lucide-react";

export const navigation = [
  { href: "/", label: "Control Tower", icon: LayoutDashboard },
  { href: "/recommendations", label: "Recommendations", icon: Lightbulb },
  { href: "/approvals", label: "Approvals", icon: CheckSquare },
  { href: "/executions", label: "Executions", icon: PlayCircle },
  { href: "/renewals", label: "Renewals", icon: FileText },
  { href: "/open-alternatives", label: "Open Alternatives", icon: RefreshCw },
  { href: "/policy-builder", label: "Policy Builder", icon: SlidersHorizontal },
  { href: "/roles", label: "Role Views", icon: Users },
  { href: "/settings", label: "Settings", icon: Settings },
] as const;

```

Sidebar + Topbar

src/components/app-shell/sidebar.tsx

```
"use client";

import Link from "next/link";
import { usePathname } from "next/navigation";
import { navigation } from "@config/navigation";
import { cn } from "@lib/utils/cn";

export function Sidebar() {
  const pathname = usePathname();

  return (
    <aside className="border-r bg-white/80 backdrop-blur">
      <div className="border-b px-6 py-5">
        <div className="text-sm font-medium text-slate-500">Autonomyx</div>
        <div className="text-lg font-semibold">Optimize</div>
      </div>
      <nav className="space-y-1 p-3">
        {navigation.map((item) => {
          const Icon = item.icon;
          const active = pathname === item.href;
          return (
            <Link
              key={item.href}
              href={item.href}
              className={cn(
                "flex items-center gap-3 rounded-2xl px-4 py-3 text-sm font-
                medium transition",
                active ? "bg-slate-900 text-white" : "text-slate-600 hover:bg-
                slate-100"
              )}
            >
              <Icon className="h-4 w-4" />
              {item.label}
            </Link>
          );
        })}
      </nav>
    </aside>
  );
}
```

src/components/app-shell/topbar.tsx

```
"use client";

import { Input } from "@/components/ui/input";

export function Topbar() {
  return (
    <div className="sticky top-0 z-10 border-b bg-white/85 px-6 py-4 backdrop-blur">
      <div className="flex items-center justify-between gap-4">
        <Input className="max-w-sm" placeholder="Search recommendations, vendors, approvals..." />
      </div>
    </div>
  );
}
```

Tenant + Role Preset Hooks

src/hooks/use-tenant.ts

```
"use client";

import { useState } from "react";

export function useTenant() {
  const [tenantId, setTenantId] = useState("t-1");
  return { tenantId, setTenantId };
}
```

src/lib/rbac/use-role-preset.ts

```
"use client";

import { useState } from "react";

export type RolePreset = "default" | "cfo" | "procurement" | "engineering";

export function useRolePreset() {
  const [rolePreset, setRolePreset] = useState<RolePreset>("default");
}
```

```
    return { rolePreset, setRolePreset };  
  }
```

Mock Seed Example

src/lib/mock/seed.ts

```
import { mockDb } from "@lib/mock/db";  
  
mockDb.recommendations = [  
  {  
    id: "rec-101",  
    title: "Reduce 180 inactive ExampleCRM seats",  
    category: "saas",  
    vendor: "ExampleCRM",  
    team: "Revenue Operations",  
    risk: "low",  
    confidence: "high",  
    status: "awaiting_approval",  
    projectedSavingsAnnual: 86400,  
    implementationCost: 2000,  
    paybackDays: 9,  
    issue: "214 inactive seats detected over 60+ days",  
    evidence: ["No login activity for 68 days"],  
    skepticNotes: ["Notify team leads before bulk license changes"],  
    rollbackPlan: ["Restore seats from latest snapshot"],  
    approvals: ["RevOps Manager"],  
    impactArea: "Revenue tooling",  
  },  
];
```

API Swap Strategy

Keep all UI pages talking only to feature hooks.

Current: - feature hook → mock repository

Later: - feature hook → REST/GraphQL client → real backend

That lets the frontend prototype become production UI with minimal page rewrite.

Production Readiness Checklist

App Foundation

- route-based pages
- reusable component library
- centralized navigation config
- typed domain models
- typed repositories

Security

- route guards
- permission helpers
- tenant scoping
- role presets separate from actual auth roles

DX

- feature-based query modules
- mock API swappable with real API
- strongly typed response contracts

Next Build Steps

- add TanStack Query provider
- add real auth provider
- add real backend client
- split mock data by domain
- add loading / empty / error states
- add server actions or API handlers for mutations
- add test coverage for RBAC and policy UI

Suggested Immediate Files to Build First

1. `src/app/(dashboard)/layout.tsx`
2. `src/config/navigation.ts`
3. `src/components/app-shell/sidebar.tsx`
4. `src/components/app-shell/topbar.tsx`
5. `src/types/domain.ts`
6. `src/lib/mock/db.ts`
7. `src/lib/mock/repositories/recommendations-repo.ts`

8. `src/lib/auth/guards.ts`
9. `src/lib/rbac/policies.ts`
10. `src/app/(dashboard)/recommendations/page.tsx`

This scaffold is designed so the existing prototype can be migrated into a real **Next.js production codebase** page by page.